Module 3 — Web Applications, HTTP and Cloud Hosting

Vistula University · IT Fundamentals · Theory handout

Vistula University · IT Fundamentals

Learning Objectives

After this module, you will be able to:

- Explain what a website is and name its two main parts: frontend and backend.
- Describe what HTML, CSS and JavaScript each do on a web page.
- Explain what a framework is and why programmers use one.
- Explain how a client talks to a server using HTTP requests and responses.
- Read a simple REST API request and a simple JSON answer.
- Explain the difference between a static and a dynamic website.
- Choose the right hosting option: GitHub Pages, Supabase, or cloud services (IaaS, PaaS, SaaS).
- Explain what a domain name and DNS are.

What Is a Website?

A **website** is a set of pages that your browser shows you. The **browser** (for example Chrome or Firefox) is a program on your computer. It asks for pages, receives them, and draws them on the screen.

In this module we use one running example: a small web app called the **Student Task Manager**. It is a page where a student can add tasks, for example "Study for the math test", set a due date, and see a list of all tasks. We will use this example for every new idea in this module.

Frontend and Backend

Every web application has two sides.

- The **frontend** is the part you see and touch. In our Task Manager, this is the page with the form, the "Add" button, and the list of tasks. The frontend runs **in your browser**.
- The **backend** is the part that works "behind the scenes". In our Task Manager, this is a program on a server that **stores the tasks**. When you close the browser and come back tomorrow, the backend still remembers your tasks. The backend runs **on a server** (a computer that is always on and always connected).

Think of a restaurant. The dining room, the menu and the waiter are the frontend. The kitchen is the backend. You never see the kitchen, but your food comes from there.

The Three Languages of the Frontend

The frontend uses three languages. Each one has a different job.

- **HTML** (HyperText Markup Language) gives the page its **structure and content**: the headings, the form fields, the buttons. It says *what is on the page*.
- **CSS** (Cascading Style Sheets) gives the page its **look**: colors, fonts, sizes, spacing. It says *how the page looks*.
- **JavaScript** gives the page its **behavior**: what happens when you click "Add". It says *what the page does*.

Here is a tiny piece of HTML for our Task Manager:

```
<h1>My Tasks</h1>
<input type="text" id="title" placeholder="Task title">
<button onclick="addTask()">Add</button>
<ul id="taskList"></ul>
```

And a small piece of CSS that makes the heading blue:

```
h1 {
  color: steelblue;
  font-family: Arial, sans-serif;
}
```

JavaScript Frameworks

Writing a big frontend in plain JavaScript is slow and repetitive. A **framework** (or **library**) is a toolbox of ready-made code that helps you build applications faster and in an organized way. Popular frontend frameworks:

- **React** — a library from Meta that builds the page from small reusable pieces called components.
- **Vue** — a framework that is easy to start with and mixes HTML-like templates with JavaScript.
- **Angular** — a full framework from Google that gives you everything in one box: structure, tools and rules.

You do not need these in this module, but you will meet them in job ads!

Node.js: JavaScript on the Server

For many years, JavaScript ran only in the browser. Then came **Node.js** — a program that runs JavaScript **outside the browser**, for example on a server. Node.js lets you write your backend in the same language as your frontend.

The difference is simple: browser JavaScript changes the page you see; Node.js JavaScript runs on a server, talks to databases, and answers requests from many users. In our Task Manager, the backend program that stores tasks could be written in Node.js.

Backend Frameworks

Backend programmers also use frameworks — toolboxes for building the server side:

- **Spring Boot** — a popular Java framework, used by many big companies.
- **Express** — a small, simple framework for Node.js, often the first backend framework students learn.
- **NestJS** — a Node.js framework with more structure and rules, good for bigger projects.

Client-Server Architecture

The web works with a simple pattern called **client-server architecture**.

- The **client** is the program that asks for something — usually your browser.
- The **server** is the program that answers — the backend.

The client sends a **request** ("give me the list of tasks"). The server sends back a **response** ("here is the list"). The client always speaks first; a server never sends data without a request. In our Task Manager: you open the page (request), the server sends the HTML file (response), and when the page loads, it asks the server for your saved tasks (another request, another response).

HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the language that clients and servers use to talk. A **protocol** is a set of rules for communication, like good manners for computers. Every request says: *what method* (read, create, update, delete), *which address*, and sometimes *some data*. Every response says: *did it work* (a status code like 200 for OK or 404 for "not found") and *the data*.

HTTPS is the same as HTTP, but the data is **encrypted** (locked with a secret code) during the trip. Nobody between you and the server can read your password or your tasks. Always prefer HTTPS — the address starts with `https://` and the browser shows a small padlock.

HTTP Methods

HTTP requests use **methods**. A method is a short word that says what you want to do. The most important ones:

Method	What it means	Example in the Task Manager
GET	Read data	Get the list of all tasks
POST	Create something new	Add a new task
PUT	Replace something completely	Replace a whole task with a new version
PATCH	Change only part of something	Mark one task as done, change nothing else
DELETE	Remove something	Delete one task

REST APIs and JSON

An **API** (Application Programming Interface) is a way for two programs to talk. A **REST API** is a web API with simple rules: every piece of data has an **address (URL)**, and you use HTTP methods to work with it. For example, our Task Manager backend could offer this address: `GET /tasks` — "give me all tasks".

The server usually answers with **JSON** (JavaScript Object Notation) — a simple text format for data. JSON is built from **name/value pairs**: a name, a colon, and a value. Here is a small JSON answer to `GET /tasks`:

```
{
  "tasks": [
    { "id": 1, "title": "Study for the math test", "done": false },
    { "id": 2, "title": "Send lab report", "done": true }
  ]
}
```

The frontend can call this API from JavaScript with the `fetch()` function:

```
fetch("https://api.example.com/tasks")
  .then(response => response.json())
  .then(data => console.log(data.tasks));
```

Hosting, Cloud, Domains and DNS

Your web page must live on a computer that is always online. Putting it there is called **hosting**. **Cloud computing** means renting computer power — servers, storage, databases — from a company instead of buying your own machines. You pay like for electricity: for what you use.

A **domain name** is the human-friendly address of a website, like `vistula.edu.pl`. Computers find each other using **IP addresses** — long numbers like `20.101.57.9`. **DNS** (Domain Name System) connects the two: it is the **phone book of the internet**. You look up the name `vistula.edu.pl`, and DNS gives you the number. Then your browser can call the right server.

Static and Dynamic Websites

- A **static website** is a fixed set of files (HTML, CSS, JavaScript). Every visitor sees the same thing. There is no server logic and no database. Good for a personal page or a simple task list saved in your own browser.
- A **dynamic website** has a backend. The server can store data, check who you are, and show different content to different users. Our full Task Manager — where tasks are saved for every user — is a dynamic web application.

Where Can You Host Your Project?

Two friendly options for students:

- **GitHub Pages** — free hosting for **static** websites, straight from your GitHub repository. Perfect for this module's lab: your `tasks.html` page will be online with one click. But it cannot store shared data on a server.
- **Supabase** — a service that gives your project a **backend**: a real database, user accounts, and a REST API, without building a server yourself. We will use Supabase in Module 4 to make tasks visible to all users.

IaaS, PaaS, SaaS

Cloud services come in three levels. You rent more or less, depending on how much you want to manage:

- **IaaS** (Infrastructure as a Service) — you rent raw machines (servers, storage) and set up everything yourself. *Example: Amazon EC2.*
- **PaaS** (Platform as a Service) — you upload your code and the platform runs it for you. *Example: Google App Engine.*
- **SaaS** (Software as a Service) — you just use the finished program in your browser. *Example: Gmail or Google Docs.*

Key Terms (Glossary)

Term	Simple meaning (English)	Po polsku
frontend	The part of a web app the user sees, running in the browser	frontend, warstwa klienta

Term	Simple meaning (English)	Po polsku
backend	The part of a web app running on a server, storing data	backend, warstwa serwerowa
HTML	Language that gives a web page structure and content	HTML (język znaczników)
CSS	Language that controls how a page looks	CSS (kaskadowe arkusze stylów)
JavaScript	Programming language that makes pages interactive	JavaScript (język skryptowy)
framework	A toolbox of ready-made code for building apps faster	framework, szkielet programistyczny
Node.js	A program that runs JavaScript on a server, outside the browser	Node.js (środowisko uruchomieniowe)
client	The program that sends requests, usually the browser	klient
server	The computer/program that answers requests	serwer
HTTP	Rules for communication between browser and server	HTTP (protokół komunikacji)
HTTPS	HTTP with encrypted (locked) data	HTTPS (szyfrowany protokół)
REST API	A web API where each resource has a URL and HTTP methods	REST API (interfejs REST)
JSON	Simple text format for data, made of name/value pairs	JSON (format wymiany danych)
DNS	The internet “phone book”: turns domain names into IP addresses	DNS (system nazw domenowych)
hosting	Storing a website on a computer that is always online	hosting (przechowywanie strony)
static website	Fixed files; every visitor sees the same content	strona statyczna
dynamic website	Has a backend; content can change per user and store data	strona dynamiczna
IaaS	Renting raw servers and managing them yourself	IaaS (infrastruktura jako usługa)
PaaS	Uploading your code to a platform that runs it	PaaS (platforma jako usługa)
SaaS	Using a finished program in the browser	SaaS (oprogramowanie jako usługa)

Summary

- A web app has a **frontend** (in the browser) and a **backend** (on a server).
- **HTML** = structure, **CSS** = style, **JavaScript** = behavior.
- Frameworks (React, Vue, Angular, Spring Boot, Express, NestJS) are toolboxes that speed up development.
- The **client sends a request** and the **server sends a response**; HTTP defines the rules, and HTTPS locks the data.
- **HTTP methods** say what you want to do: GET, POST, PUT, PATCH, DELETE.
- A **REST API** gives data an address; the data usually travels as **JSON**.
- **DNS** turns a domain name into an IP address, like a phone book.
- **Static** sites are fixed files (hosted free on GitHub Pages); **dynamic** apps need a backend (like Supabase).
- Cloud services: **IaaS** (rent machines), **PaaS** (rent a platform), **SaaS** (use a ready program).